

Labo 00 — Praktijklabo: een rondleiding door Linux vanuit de terminal

Doel: elk begrip uit de theorie zelf in handen krijgen — processen, signalen, exitcodes, permissies, omgeving, stromen, poorten, archieven — met niets meer dan wat Ubuntu 24.04 standaard aan boord heeft. Containers komen hier nog niet aan bod, maar alles wat je hier doet, komt ongewijzigd terug in de Docker-labo's.

Vereisten — Windows 10/11 met WSL 2 en een **Ubuntu 24.04**-distributie. Meer niet: nog geen Podman (dat is labo 01), geen extra pakketten. Open een Ubuntu-terminal en blijf erin werken.

Stap 0 — Waar ben ik, en wie ben ik?

```
head -n 2 /etc/os-release
uname -r
whoami
id
```

Observeer: `PRETTY_NAME="Ubuntu 24.04.x LTS"`; een kernel zoals `6.6.87.2-microsoft-standard-WSL2` (het achtervoegsel is de WSL-handtekening); je gebruikersnaam; en een regel `uid=1000(...)` `gid=1000(...)` `groups=... 27(sudo) ...`.

Uitleg. Je zag net drie identiteiten die je nooit meer mag verwarren: de **distributie** (Ubuntu 24.04 — de userland), de **kernel** (door Microsoft gebouwd voor WSL) en **jijzelf** (UID 1000, lid van de groep `sudo`). Voor de kernel ben je niets meer dan dat nummer: 1000.

→ WINDOWS / WSL

Eindigt `uname -r` niet op `-microsoft-standard-WSL2`, dan zit je niet in WSL 2. Controleer vanuit PowerShell met `wsl --version` en `wsl --list --verbose`. De hele laboreeks gaat uit van WSL 2.

Stap 1 — De kernel, de userland en de grens ertussen

```
cat /proc/version
type ls
type cd
which cat
```

Observeer: de kernelversie voluit; `ls` is `/usr/bin/ls` — een programma, een bestand op schijf; `cd` is a shell builtin — helemaal geen programma, maar een functie binnen de shell; en `/usr/bin/cat`.

Uitleg. Alles wat je typt is ofwel een **programma** uit de userland (een uitvoerbaar bestand ergens op schijf), ofwel een builtin van de shell. Geen van beide raakt de hardware aan — allebei gaan ze via de system calls van de kernel. En dat `cd` een builtin is, heeft een goede reden: de huidige map is een eigenschap *van het shellproces zelf*. Een extern `cd`-programma zou zijn eigen map wijzigen en dan stoppen — jouw shell zou er niets van merken.

→ LINUX

`type` vraagt aan de shell "wat zou jij met dit woord doen?"; `which` zoekt alleen in het `PATH`. Lijkt een commando je te bedriegen (een alias, een functie), vertrouw dan op `type` — dat zegt altijd de waarheid.

Stap 2 — Eén boom, veel mounts

```
ls /  
findmnt /  
df -h /  
ls /mnt/c/Windows 2>/dev/null | head -n 3
```

Observeer: één enkele wortel (`bin boot dev etc home ... proc ... tmp usr var`); een `findmnt`-regel zoals `/ /dev/sdc ext4 rw,relatime,...`; een schijfgrootte rond `1007G` (de *virtuele* grootte van de WSL-schijf); en — dit is Windows, gezien vanuit Linux — de inhoud van `C:\Windows`.

```
findmnt -t proc  
ls /proc | head -n 8  
grep MemTotal /proc/meminfo
```

Observeer: `/proc proc proc rw,relatime` — een bestandssysteem van het type `proc`, zonder schijf erachter. De `ls` toont **getallen**, één per levend proces, en `MemTotal` komt rechtstreeks van de kernel.

Uitleg. Geen `C:` - of `D:` -stations hier. Alles hangt aan één boom, vastgemaakt met **mounts**: de Linux-schijf levert `/`, de Windows-schijf hangt aan `/mnt/c`, en de kernel zelf hangt aan `/proc` — een map waarvan de bestanden bij elke lezing opnieuw worden gegenereerd. Docker-images (labo 02) en volumes (labo 06) zijn in wezen niets meer dan extra mounts aan deze boom.

→ WINDOWS / WSL

Elke toegang tot `/mnt/c` steekt de grens Windows ↔ Linux over, en dat is **traag**. Projecten die je compileert en images die je opslaat, horen aan de Linux-kant (`/home/...`), niet in `/mnt/c/Users/...`. Maak daar nu al een gewoonte van.

Stap 3 — Processen: PID, ouder, /proc

```
ps  
echo $$  
ps -p 1 -o pid,comm
```

Observeer: `ps` toont bijna niets (jouw `bash` en de `ps` zelf); `echo $$` drukt het PID van je shell af; en proces 1 is `systemd`. Op zichzelf toont `ps` alleen de processen van *jouw terminal*. Alle andere — `ps -ef` toont ze wél — draaien zonder terminal, en de meeste daarvan zijn **daemons**: dienstprocessen zoals `systemd` zelf, met namen die meestal op "d" eindigen.

Start nu een proces dat even blijft hangen, op de achtergrond:

```
sleep 300 &
ps -o pid,ppid,stat,cmd
```

Observeer een regel `sleep 300` waarvan het **PPID gelijk is aan het PID van jouw bash** — een ouder-kindrelatie, live:

```
PID  PPID  STAT  CMD
2363 2362  S     bash
2419 2363  S     sleep 300
2420 2363  R     ps -o pid,ppid,stat,cmd
```

Ga dat proces nu bekijken in `/proc` (vervang `2419` door jouw eigen PID):

```
head -n 3 /proc/2419/status
tr '\0' ' ' < /proc/2419/cmdline; echo
ls -l /proc/2419/exe
```

Observeer: `Name: sleep`, `State: S (sleeping)`, de exacte commandoregel, en een link `exe -> /usr/bin/sleep`.

Uitleg. Er zit geen magie in `ps` — het leest gewoon `/proc`. Alles wat Docker je later toont (`podman top`, `podman inspect`) komt uit dezelfde bron. `STAT S` betekent *sleeping* (wachtend); `R` betekent *running*.

Stap 4 — Signalen en exitcodes

De `sleep` draait nog. Vraag hem vriendelijk om te vertrekken:

```
kill 2419          # jouw eigen PID hier
ps -o pid,cmd | grep "[s]leep 300" || echo "geen sleep-proces meer"
```

Observeer: de shell meldt `Terminated`, daarna `geen sleep-proces meer`. Een gewone `kill` stuurt `SIGTERM`, en `sleep` gehoorzaamt.

Nog een keer, maar nu hardhandig:

```
sleep 300 &
kill -9 %1
```

Observeer: deze keer meldt de shell `Killed`. `SIGKILL` heeft niets gevraagd. (`%1` verwijst naar *job* nummer 1 van de shell — handig als je geen zin hebt om het PID op te zoeken.)

Verzamel nu de exitcodes:

```
true; echo $?
false; echo $?
ls /bestaat-niet; echo $?
onbekend-commando; echo $?
bash -c 'kill -9 $$'; echo $?
```

Observeer, in volgorde: `0`, `1`, `2` (na de foutmelding van `ls`), `127` (na `command not found`) en `137` (na `Killed`).

Uitleg. `0` betekent succes, al de rest is een mislukking, en `128 + n` betekent dood door signaal n — dus `137 = 128 + 9 =` gedood door SIGKILL. Deze vijf getallen zijn exact wat `podman ps` in labo 03 in zijn kolom `Exited (...)` zal tonen. Leer ze hier lezen, waar alles nog eenvoudig is.

ONTHOUDEN

Escaleer in de juiste volgorde: eerst `kill` (SIGTERM — de applicatie mag opruimen), dan wachten, en pas daarna `kill -9` (SIGKILL — de kernel veegt weg). `docker stop` voert dat protocol voor jou uit: SIGTERM, tien seconden gratie, dan SIGKILL.

Stap 5 — De omgeving en het PATH

```
echo $HOME
env | wc -l
env | grep -E '^(HOME|PATH|LANG)='
```

Observeer je omgeving: enkele tientallen variabelen, waaronder `HOME=/home/<jij>` en een `PATH` dat op WSL zelfs Windows-paden bevat (`/mnt/c/Windows/system32 ...`).

Nu het beslissende experiment — een shellvariabele is **geen** omgevingsvariabele:

```
MSG=hallo
echo $MSG
bash -c 'echo kind ziet: [$MSG]'
export MSG
bash -c 'echo kind ziet: [$MSG]'
```

Observeer: `hallo`, dan `kind ziet: []` — leeg! — en na de `export`: `kind ziet: [hallo]`.

Uitleg. Elk kindproces krijgt een **kopie** van de omgeving van zijn ouder, bevroren op het moment van de start. Vóór de `export` bestond `MSG` alleen in jouw shell. Dit is exact het mechanisme waarmee `podman run -e MSG=hallo` in labo 08 jouw applicaties zal configureren.

Dan het `PATH`:

```
mkdir -p ~/labo0/tools
printf '#!/bin/bash\nnecho "eigen tool: ok"\n' > ~/labo0/tools/mijntool
chmod +x ~/labo0/tools/mijntool
mijntool; echo $?
export PATH="$HOME/labo0/tools:$PATH"
mijntool
```

Observeer: eerst `command not found` en `127`; zodra de map in het `PATH` staat: `eigen tool: ok`.

Uitleg. De shell "kent" geen enkel commando. Hij doorzoekt de mappen van het `PATH` in volgorde op een uitvoerbaar bestand met die naam, en stopt bij de eerste treffer. (Dit aangepaste `PATH` geldt alleen voor deze shell — blijvend maak je het met één regel in `~/.bashrc`.)

Stap 6 — Drie stromen: redirections en pipes

```
cd ~/labo0
echo "eerste regel" > notities.txt
echo "tweede regel" >> notities.txt
cat notities.txt
```

Observeer: `>` maakt aan (of overschrijft!), `>>` voegt toe.

Splits nu de twee uitvoerstromen:

```
ls notities.txt /bestaat-niet > uitvoer.txt 2> fouten.txt
cat uitvoer.txt
cat fouten.txt
```

Observeer: het scherm bleef stil tijdens de `ls`. In `uitvoer.txt` staat `notities.txt`; in `fouten.txt` staat `ls: cannot access '/bestaat-niet': No such file or directory`.

```
ls notities.txt /bestaat-niet > alles.txt 2>&1
cat alles.txt
```

Observeer beide regels samen: `2>&1` sluit de foutstroom (2) aan op de plek waar de uitvoerstroom (1) op dat moment naartoe wijst.

Tot slot de pipes:

```
ps -ef | wc -l
ps -ef | grep "[bash]" | head -n 3
```

Observeer het aantal processen op het systeem, en daarna jouw shells — zonder ergens een tussenbestand: de uitvoer van elk commando voedt de invoer van het volgende.

→ LINUX / SHELL

Over die truc met `grep "[bash]"`: de haakjes vormen een reguliere expressie die `bash` matcht, maar op de commandoregel van de `grep` zelf staat `[bash]`, en dat matcht niet met het patroon. Zonder deze truc zou `grep` zichzelf altijd terugvinden in de lijst. Dit idioom zie je in elk labo terug.

Stap 7 — Permissies: een `ls -l` leren lezen

```
ls -l notities.txt
stat -c "%U %G %a %n" notities.txt
```

Observeer: `-rw-r--r-- 1 <jij> <jij> 27 ... notities.txt`, en de numerieke vorm `644` — eigenaar `rw` (6), groep `r` (4), anderen `r` (4).

Maak een script en probeer het uit te voeren:

```
printf '#!/bin/bash\nnecho "Hallo, ik ben proces $$"\n' > hallo.sh
./hallo.sh; echo $?
chmod +x hallo.sh
ls -l hallo.sh
./hallo.sh
```

Observeer: `Permission denied` en code **126** — gevonden, maar niet uitvoerbaar. Na `chmod +x : -rwxr-xr-x`, en het script draait — telkens met een ander PID.

Nu de grens met root:

```
cat /etc/shadow; echo $?
ls -l /etc/shadow
sudo head -n 1 /etc/shadow
```

Observeer: `Permission denied` (code 1); de regel `-rw-r----- 1 root shadow ...` die dat verklaart (je bent geen `root` en zit niet in de groep `shadow`); en via `sudo` de eerste regel `root:!:...` (`*` of `!` betekent: account vergrendeld, geen enkel wachtwoord past ooit).

Uitleg. De kernel legt de UID van het proces naast de drie `rwX`-tripletten en past het eerste toe dat op jou van toepassing is. `sudo` sluipt nergens omheen: het start het proces met UID 0, en UID 0 wordt door de kernel niets geweigerd. Hou dit model bij de hand voor labo 06, wanneer een container bestanden in een volume schrijft onder een onverwachte UID.

Stap 8 — Een server, een poort, een client

Ubuntu 24.04 levert Python mee, dus je eerste HTTP-server is één regel ver.

```
echo "<h1>Hallo vanaf mijn server</h1>" > index.html
python3 -m http.server 8080 &
curl -s http://localhost:8080/index.html
```

Observeer je HTML die via HTTP terugkomt: `<h1>Hallo vanaf mijn server</h1>`.

```
curl -si http://localhost:8080/index.html | head -n 4
ss -tlnp | grep 8080
```

Observeer het volledige HTTP-antwoord (`HTTP/1.0 200 OK`, `Server: SimpleHTTP/0.6 Python/3.12.3`, `Content-type: text/html`) en de luisterende socket:

```
LISTEN 0 5 0.0.0.0:8080 0.0.0.0:* users:(("python3",pid=2788,fd=3))
```

`0.0.0.0:8080`: het proces `python3` luistert op **alle** interfaces, poort 8080.

→ **WINDOWS / WSL**

Open een **Windows**-browser op `http://localhost:8080` — de pagina laadt. WSL 2 stuurt `localhost` automatisch door van Windows naar Ubuntu. In labo 07 test je dankzij diezelfde doorschakeling je containers vanuit een Windows-browser.

Probeer nu een geprivilegieerde poort:

```
python3 -m http.server 80
```

Observeer de mislukking: `PermissionError: [Errno 13] Permission denied`. Poort 80 ligt onder de drempel van 1024, en die zone is van root — jij bent UID 1000. Rootless Podman leeft met dezelfde beperking.

Zet de server uit en controleer:

```
kill %1  
curl -s --max-time 2 http://localhost:8080/; echo $?
```

Observeer exitcode **7** van `curl` — *connection refused*. Er luistert niemand meer.

Stap 9 — Archieven: `tar`, de voorloper van images

```
mkdir -p mijn-app/config  
echo "app.port=8080" > mijn-app/config/app.properties  
echo "namaakbinary" > mijn-app/app.bin  
tar -czf mijn-app.tar.gz mijn-app  
ls -lh mijn-app.tar.gz  
file mijn-app.tar.gz
```

Observeer een archief van enkele honderden bytes, herkend als `gzip compressed data`.

```
tar -tf mijn-app.tar.gz  
mkdir -p /tmp/herstel  
tar -xzf mijn-app.tar.gz -C /tmp/herstel  
cat /tmp/herstel/mijn-app/config/app.properties
```

Observeer de inhoudslijst (`-t` staat voor *list*), daarna de extractie naar een andere plek (`-C`), en het bestand dat er weer exact zo uitkomt: `app.port=8080`.

Uitleg. `tar` (*tape archive*, 1979) stopt een volledige mappenboom — paden, permissies, eigenaars — in één bestand. Onthoud dat goed: een **layer** van een Docker-image is letterlijk een tar-archief, en `podman save` (labo 02) geeft je straks een tar vol tars. Niets nieuws onder de zon.

Opruimen

Controleer dat er geen laboproces meer rondhangt, en verwijder daarna de bestanden:

```
ps -o pid,cmd | grep -E "[s]leep|[h]ttp.server" || echo "niets meer te doden"  
rm -r ~/labo0  
rm -r /tmp/herstel
```

Het aangepaste `PATH` en de variabele `MSG` sterven mee met deze shell — de terminal sluiten volstaat. Er werd niets geïnstalleerd, dus er valt ook niets te verwijderen.

Wat je nu moet kunnen zeggen

- Mijn kernel is `...-microsoft-standard-WSL2`; mijn distributie is Ubuntu 24.04; ik ben UID 1000.
- Een proces heeft een PID en een ouder. Ik zag er een geboren worden (`&`), leven (`/proc/<pid>/`) en sterven (`kill`).
- `kill` stuurt SIGTERM, waarover te praten valt; `kill -9` stuurt SIGKILL, waarover niet. Een proces dat door SIGKILL sterft, eindigt met `137` = `128 + 9`.
- `$?` is `0` bij succes; `126` betekent niet uitvoerbaar; `127` betekent niet gevonden in het `PATH`.
- Een variabele bereikt kindprocessen pas na `export` — en een proces dat al draait, bereikt ze nooit.
- `>` vangt stdout op, `2>` stderr, `2>&1` voegt ze samen, `|` schakelt processen achter elkaar.
- `-rw-r-----` lees je als drie tripletten; de kernel vergelijkt UID's; `root` (UID 0) slaat de controles gewoon over.
- `ss -tlnp` toont wie op welke poort luistert; `0.0.0.0` betekent alle interfaces; onder 1024 is het domein van root; met `curl` test je het allemaal.
- Een mount maakt een bestandssysteem vast aan de ene boom; `/proc` heeft geen schijf; `tar` verpakt een boom — precies wat Docker-images ook doen.